

FS Module Study Notes

Sync • Async Callbacks • Promises • Path Basics

Prepared for Junaid

Quick Revision Goal

Ye notes tumhari ab tak ki Node.js practice ke according hain. Focus: path + fs module ko 3 approaches mein samajhna: synchronous, callback-based asynchronous, aur promise-based asynchronous.

1. Importing path and fs

```
const path = require("path");  
const fs = require("fs");
```

Node.js ke built-in modules ko CommonJS mein require() ke through import kiya ja sakta hai.

2. Building a Full File Path

```
const filePath = path.join(__dirname, "Ays.text");
```

- path.join() different path parts ko safely combine karta hai.
- __dirname current JavaScript file ke folder ka full path deta hai.

3. FS Module ke 3 Main Approaches

A. Synchronous FS

Synchronous methods ke naam ke end par usually Sync hota hai. Inmein Node.js operation complete hone tak next line par nahi jata.

```
fs.writeFileSync()
fs.readFileSync()
fs.appendFileSync()
fs.unlinkSync()
fs.existsSync()
```

```
fs.writeFileSync(filePath, "Hello", "utf-8");
```

Easy Memory Trick

SYNC: Operation complete -> phir next line execute hoti hai.

B. Asynchronous FS - Callback

Async callback methods operation ko start karte hain aur result/error milne par callback function execute hota hai.

```
fs.writeFile()
fs.readFile()
fs.appendFile()
fs.unlink()
```

Write / Create File

```
fs.writeFile(filePath, "Welcome Async FS System", "utf-8", (err) => {
  if (err) {
    console.log("Error");
  } else {
    console.log("File Successfully Created");
  }
});
```

Read File

```
fs.readFile(filePath, (error, data) => {
  if (error) {
    console.log("Error");
  } else {
    console.log(data.toString());
  }
});
```

Agar encoding directly nahi dete to `readFile()` Buffer de sakta hai. `data.toString()` se Buffer ko readable string mein convert kar sakte ho.

```
fs.readFile(filePath, "utf-8", (err, data) => {  
  console.log(data);  
});
```

Append / Update File

```
fs.appendFile(filePath, "\nNew Data", "utf-8", (err) => {  
  if (err) {  
    console.log(err);  
  } else {  
    console.log("Data Updated");  
  }  
});
```

Important: \n

New line ke liye string ke start mein `\n` use karo. Iske baghair purana aur naya data same line par jud sakta hai.

Delete File

```
fs.unlink(filePath, (err) => {  
  if (err) {  
    console.log(err);  
  } else {  
    console.log("File Deleted");  
  }  
});
```

C. Promise-based FS

Promise-based FS mein success ko `.then()` aur error ko `.catch()` se handle kiya ja sakta hai.

```
fs.promises.writeFile()  
fs.promises.readFile()  
fs.promises.appendFile()  
fs.promises.unlink()
```

```
fs.promises.someMethod()
  .then(() => {
    console.log("Success");
  })
  .catch((err) => {
    console.log(err);
  });
```

Promise - Write File

```
fs.promises
  .writeFile(filePath, "Initial Data", "utf-8")
  .then(() => console.log("File Successfully Created"))
  .catch((err) => console.log(err));
```

Promise - Read File

```
fs.promises
  .readFile(filePath, "utf-8")
  .then((data) => console.log(data))
  .catch((err) => console.log(err));
```

Promise - Append File

```
fs.promises
  .appendFile(filePath, "\nNew Data", "utf-8")
  .then(() => console.log("Data Added"))
  .catch((err) => console.log(err));
```

4. Important Promise Mistake

```
// Wrong
.then(console.log("File Created"))

// Correct
.then(() => console.log("File Created"))
```

Reason: `console.log("File Created")` direct call hai aur turant execute ho sakta hai. `() => console.log("File Created")` ek function hai jo Promise successful hone ke baad call hota hai.

5. Sync vs Callback vs Promise

Type	Example	Main Idea
Sync	<code>readFileSync()</code>	Code waits for operation
Callback Async	<code>readFile()</code>	Callback handles result/error
Promise Async	<code>fs.promises.readFile()</code>	<code>.then()</code> / <code>.catch()</code>

Easy Memory Trick

SYNC -> operation complete -> next line

CALLBACK -> operation complete -> callback

PROMISE -> success -> `.then()` | error -> `.catch()`

6. Your Current Node.js Coverage

Modules

- `require`
- `import`
- `export`
- single value
- array
- object
- destructuring

Path Module

- `join`
- `dirname`
- `basename`
- `extname`
- `parse`
- `sep`
- `__dirname`
- `__filename`

OS Module

- basic system / OS information

FS Module

- synchronous methods
- asynchronous callbacks

- promise-based methods

7. Important Practice Notes

- File path ke liye `path.join(__dirname, filename)` use karna strong habit hai.
- Append karte waqt new line chahiye ho to `\n` add karna yaad rakho.
- Callback mein error aur success dono cases handle karo.
- Promise mein `.then(() => ...)` ke andar function pass karo; `console.log()` ko direct call na karo.
- Multiple async operations ka fixed order chahiye ho to unko properly chain karna hota hai.

END OF NOTES